Claims Investigation Copilot — Agentic Tree

```
graph TD
    %% ── Entry ──
    USER(["👤 User / API<br/>Initial fraud query"])
    DATA["📦 Data Foundation<br/>main.py → DataContext<br/>50K claims · 500 providers · 5K members"]

    USER -->|trigger| RUNNER["▶ run_agents.py / api.py<br/>Initialize data + set context"]
    RUNNER --> DATA
    RUNNER -->|start graph| ORCH

    %% ── Orchestrator ──
    ORCH["🎯 Orchestrator Agent<br/><i>Lead Investigator — no tools</i><br/>Routes phases: scan → investigate → compile → done"]

    %% ── Routing ──
    ORCH -->|"NEXT_PHASE: INVESTIGATE"| INV
    ORCH -->|"NEXT_PHASE: COMPILE"| DOS
    ORCH -->|"NEXT_PHASE: DONE"| DONE(["✅ Investigation Complete<br/>Dossier saved"])

    %% ── Investigation Agent ──
    INV["🔍 Investigation Agent<br/><i>Detective — 7 tools</i><br/>AWS Bedrock Claude"]
    INV --> ORCH

    INV --- T1["🔧 scan_new_claims<br/>High-anomaly entity scan"]
    INV --- T2["🔧 profile_entity<br/>Entity detail profile"]
    INV --- T3["🔧 compare_to_peers<br/>Statistical z-score comparison"]
    INV --- T4["🔧 get_claim_details<br/>Raw claim records"]
    INV --- T5["🔧 find_connections<br/>NetworkX graph traversal"]
    INV --- T6["🔧 find_ring<br/>Fraud ring detection"]
    INV --- T7["🔧 get_referral_history<br/>Referral pattern analysis"]

    %% ── Dossier Agent ──
    DOS["📝 Dossier Agent<br/><i>Case Writer — 5 tools</i><br/>AWS Bedrock Claude"]
    DOS --> ORCH

    DOS --- D1["🔧 assess_evidence<br/>Sufficiency check"]
    DOS --- D2["🔧 search_billing_rules<br/>FAISS vector search<br/>CMS / OIG rules"]
    DOS --- D3["🔧 find_similar_cases<br/>Precedent case lookup"]
    DOS --- D4["🔧 estimate_recovery<br/>Recoverable $ calculation"]
    DOS --- D5["🔧 compile_dossier<br/>Final markdown report"]

    %% ── Data Layer ──
    subgraph DATA_LAYER["Data & ML Layer"]
        SYN["generate_synthetic.py<br/>50K claims generator"]
        FEAT["features.py<br/>Provider & member features"]
        ANOM["anomaly.py<br/>Isolation Forest ML"]
        GRF["graph.py<br/>NetworkX directed graph"]
        RULES["billing_rules/<br/>FAISS + rules.json"]
    end

    DATA --- SYN
    DATA --- FEAT
    DATA --- ANOM
    DATA --- GRF
    DATA --- RULES

    %% ── Frontend ──
    subgraph FRONTEND["React Frontend"]
        WS["WebSocket streaming"]
        DASH["Dashboard UI"]
        NET["Network Graph Viz"]
        TL["Event Timeline"]
        DP["Dossier Panel"]
    end

    RUNNER -.->|FastAPI + WS| WS
    WS --> DASH
    WS --> NET
    WS --> TL
    WS --> DP

    %% ── Styles ──
    classDef agent fill:#4A90D9,stroke:#2C5F8A,color:#fff,font-weight:bold
    classDef tool fill:#F5A623,stroke:#C77D0A,color:#fff
    classDef data fill:#7ED321,stroke:#4A8C12,color:#fff
    classDef frontend fill:#BD10E0,stroke:#8B0BA6,color:#fff
```

```
    classDef entry fill:#D0021B,stroke:#9B0114,color:#fff

    class ORCH,INV,DOS agent
    class T1,T2,T3,T4,T5,T6,T7,D1,D2,D3,D4,D5 tool
    class DATA,SYN,FEAT,ANOM,GRF,RULES data
    class WS,DASH,NET,TL,DP frontend
    class USER,DONE entry
```